Contracts Across the Multi-Robot Execution Stack

A Cross-Layer Review and Reference Framework

Mohamed Adil Farukh

mdadilfarooq@gmail.com

Overview

A robot may receive a plan, prediction, or state update that was valid when it was produced but is no longer safe to use. What should travel with that result so the receiving component can check it? In this note, I review contract-based and runtime-assurance approaches with that question in mind. I organize the review by what each approach protects and identify five recurring interface gaps: meaning and physical validity; timing; uncertainty and evidence; implementation and lifecycle compatibility; and recovery authority. Public failure reports from the ROS 2 navigation stack illustrate the compatibility and lifecycle gap.

I then propose an interface-contract framework that records the payload, metadata, assumptions, guarantees, checks, evidence, and recovery rule for a transfer. Required checks must pass before the receiver uses the result; missing evidence does not count as a pass. A worked collision-avoidance condition shows how state age, clock error, and estimation error consume the available margin, and when a local estimate can be preferable to a more accurate remote one. A command trace and failure taxonomy show how the fields fit together. The central finding is that validity depends on more than the payload: the receiver also needs the timing, assumptions, and evidence behind it. The framework turns those dependencies into explicit interface checks.

Keywords: Multi-robot systems, assume–guarantee contracts, runtime assurance, runtime verification, ROS 2, age of information.

1 The question

A plan or prediction is useful only while its assumptions hold. In a multi-robot system, the component using a result may be several steps away from the component that produced it. It may even be on another robot, with a different clock and an older view of the world. The question behind this note is simple: *what should travel with a result so the receiver can decide whether it is still valid?*

A mission planner, motion predictor, middleware layer, and safety controller answer different questions. A mission synthesizer may guarantee a temporal goal within a declared context. A planner may find a feasible route for a particular state and model. A predictor may forecast peer motion over a horizon. None of these claims automatically tells the next component whether its own assumptions are satisfied.

This matters as robot stacks combine task allocation and motion coordination [1–3] with language-conditioned planning and learned prediction [4–6]. The newer components still depend on physical, timing, and deployment assumptions. Where inference runs adds another tradeoff: an on-board estimate is close to the controller, while a remote estimate may use more data but arrive later [7].

At lower layers, ROS 2 and the Data Distribution Service (DDS) carry state and commands, and runtime-assurance or barrier-based controllers may override a decision before actuation [8–11].

The multi-robot case adds a practical difficulty. Robot r_i can make a locally reasonable decision using stale peer state and invalidate robot r_j's route, resource claim, or safety margin. Joining, leaving, restarting, and recovering from a network partition can also invalidate assignments while messages remain syntactically correct. I distinguish *vertical* interfaces within a robot from *horizontal* interfaces between robots or distributed services (Fig. 1).

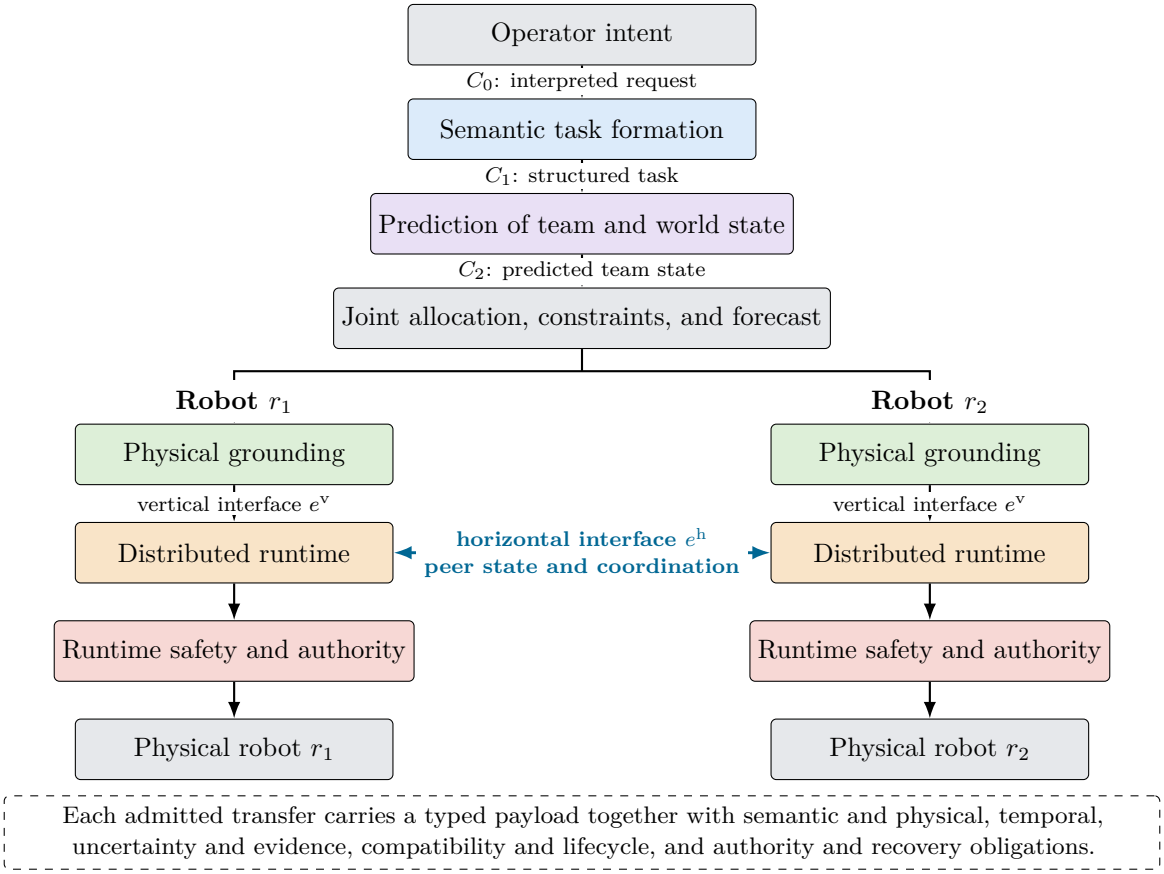


Figure 1: Cross-layer command path of a two-robot team. Single-headed arrows are producer–consumer transfers; C_0–C_2 label the shared command-formulation interfaces. Vertical interfaces connect functions within one robot, and the double-headed horizontal interface exchanges peer state and coordination obligations between robots.

Existing work addresses parts of this question through mission contracts, quantitative mission assurance, component contracts for the Robot Operating System (ROS), compositional reasoning for dynamics, monitoring, and safety switching [10, 12–16]. I reviewed these approaches with three questions in mind:

1. What does each approach protect, and which assumptions and guarantees does it represent?
2. How are those claims combined, checked, enforced, or repaired, and what evidence supports them?
3. What remains implicit when a result crosses a component or robot boundary, and what would make those assumptions checkable?

What this note contains

I group the reviewed approaches by what they protect, then compare their coverage of interface obligations (Sections 4 and 5). From the gaps, I propose a reference framework for recording validity checks, evidence, and recovery rules (Section 6). A temporal admission condition for peer state makes the timing and estimation fields concrete and shows when an on-board estimate can be preferable to a more accurate remote estimate. A failure taxonomy and command trace show how to assign checks and recovery authority along the command path (Sections 6.6 and 6.7).

The useful connection is between interface design and the decision that uses the data. The peer-state calculation shows why a time stamp, a clock-error bound, or an estimation-error bound belongs in an interface: each changes the margin available to the receiving robot.

2 The Multi-Robot Interface Problem

2.1 Coupled Execution

A multi-robot system contains a team $\mathcal{R} = \{r_1, \ldots, r_n\}$ whose objectives, observations, actions, or resources are coupled [1, 17, 18]. The coupling may arise from a shared mission, collision constraints, task precedence, limited resources, or communication over a time-varying graph. Local correctness is therefore insufficient in general: a decision that is safe for one robot under an old peer state can violate a joint constraint under the current team state.

Let x_i, u_i, and w_i denote the state, input, and disturbance of robot r_i. A general model is

$$\dot{x}_i = f_i(x_i, u_i, w_i), \qquad g_{ij}(x_i, x_j) \geq 0, \quad \{i, j\} \in \mathcal{E}_c, \tag{1}$$

where $\mathcal{E}_c$ indexes the coupled team constraints. For collision avoidance, $g_{ij} = \|p_i - p_j\|^2 - d_{\min}^2$ for positions p_i, p_j and minimum separation $d_{\min}$; other instances encode formation, visibility, or connectivity constraints [11, 15, 19]. The local dynamics in (1) can be modeled correctly while the joint constraint is evaluated with an invalid peer state. This is the mechanical reason why horizontal interfaces need explicit timing and uncertainty assumptions.

A command typically passes through task interpretation, allocation, prediction, motion or action planning, distributed communication, runtime validation, and physical execution. These functions need not occupy separate processes or robots, but each transformation changes the object being reasoned about. A language instruction becomes a task model, a task model becomes an assignment, an assignment becomes trajectories, and distributed observations become inputs to safety-critical control. Evidence that is meaningful at one stage may be insufficient at the next.

2.2 Contracts, Verification, and Assurance

An assume–guarantee contract separates an environment assumption A from a component guarantee G [20]. A contract theory additionally defines satisfaction, compatibility, refinement, composition, and quotient for a chosen behavioral model. Robotics uses the idea at several levels: a mission contract describes temporal goals and contexts, a component contract specifies software behavior, a dynamical contract constrains input and output trajectories, and a runtime-assurance specification governs the transfer of control authority. These contract objects are related but not interchangeable.

I use three terms consistently. A *contract* states obligations, *verification* checks a stated property, and *assurance* is the broader argument and evidence supporting confidence in system behavior. A contract contributes to assurance only when its assumptions, checking mechanism, and evidence are explicit. An interface schema is therefore not automatically a contract. Names, types, time stamps, and version fields become contractual only when they appear in explicit assumptions or guarantees, are checked by a defined validator, and have a specified consequence when the check fails. Likewise, a runtime monitor observes only the properties encoded in its specification; it cannot infer an omitted frame, validity horizon, or recovery policy.

2.3 Kinds of Evidence

Because every guarantee is conditional on its assumptions, the kind of supporting evidence matters. I distinguish three kinds. *Capability evidence* shows that a system performs a task. *Assurance evidence* supports a specified property under declared assumptions, for example through proof, synthesis, monitoring, or systematic testing. *Deployment evidence* measures behavior on a stated platform, configuration, workload, or fault condition. A planning benchmark is not evidence of robustness to network timing, and a formal guarantee under a dynamics model is not evidence that an implementation uses the correct frame, clock, or software version. The distinction matters for coordinated-autonomy studies in particular: a mission-level probability or reliability estimate can be valid for its model and data without certifying the runtime interfaces that execute the mission [21, 22].

3 How I reviewed the literature

I used a structured narrative review to examine how contract-based and runtime-assurance approaches handle assumptions across software and robot interfaces. For each source, I asked four questions:

1. What is the primary object of reasoning: mission, component behavior, dynamical interaction, distributed state, runtime authority, or embodied execution?
2. Which mechanism does the source supply: specification, refinement, composition, synthesis, monitoring, testing, intervention, or repair?
3. Which kind of evidence does it report (Section 2.3), and does that evidence include a physical multi-robot system?
4. Which semantic, physical, temporal, uncertainty, evidence, compatibility, lifecycle, and authority obligations does it leave to adjacent components?

Search. I searched arXiv and web-indexed IEEE, ACM, Springer, Elsevier, and institutional repositories by title, abstract, and full text. Core queries combined *assume–guarantee* or *contract-based* with *robot*, *ROS*, *multi-robot*, *multi-agent*, *mission specification*, *coordinated autonomy*, *mission reliability*, *probabilistic model checking*, *runtime verification*, and *runtime assurance*. Following references and later citations from foundational contract work, coordinated-mission assurance, robotic mission frameworks, ROS contract and monitoring systems, and recent cross-layer reviews added sources that use different terminology.

Selection. I included primary work that defines a contract, compositional-assurance, runtime-monitoring, or runtime-assurance mechanism applicable to robotics, cyber-physical systems,

or interconnected agents; reports a relevant robotics implementation; or defines a directly relevant communication standard. I also included surveys that organize multi-robot modeling, communication, learning, safety, or embodied assurance. I excluded general design-by-contract work without an architectural or cyber-physical connection and planning work that does not expose an interface-assurance question. Representative planning and middleware sources were retained only where they clarify what an interface must carry.

Coding. For each source, I recorded the protected object, mechanism, reported evidence, deployment setting, and obligations left to adjacent components, and I grouped sources into twelve approach classes under six protected objects (Table 1). The coded corpus contains 36 sources. The remaining 15 references supply background on multi-robot coordination and middleware, foundational contract theory, the standards used by the framework, related surveys, and the public issue reports analyzed in Section 5.6. Coverage of each obligation is coded per class (Table 2). A class receives "E" when the obligation is explicit and central in every source of the class, "P" when the treatment is partial, conditional, model-internal, or present in only some sources, and "–" when the obligation lies outside the stated mechanisms. The coding describes the cited mechanisms, not every capability of the surrounding systems.

Scope of inference. The search grew as I followed references and citations, so PRISMA-style candidate and exclusion counts were not retained. The review therefore supports qualitative statements about coverage and evidence boundaries; it supports no claim about prevalence or effect size.

4 What the reviewed approaches protect

I organize the reviewed literature by its *protected object*: the property or artifact that an approach is designed to preserve. Table 1 summarizes the twelve approach classes; the classes overlap, and one system can contribute to more than one of them.

Table 1: Reviewed approaches grouped by what they protect

Approach class	Protected object	Mechanism	Reported evidence	Obligations left to adjacent components
Mission intent				
Mission contracts [12, 23, 24]	Goals, contexts, and component libraries	Contract-based goal graphs, refinement, synthesis, context switching, and repair	Mission examples and synthesized controllers; no physical team evaluation	Physical validity, event-time state, implementation identity, and peer reconciliation
Language-conditioned planning [4, 5]	Instructions, roles, task dependencies, and plans	Dialogue with environment feedback; dependency graph with a linear program	Multi-robot benchmarks, simulation, and selected real-world execution	Calibrated uncertainty, timing, compatibility, and deployment assurance
Coordinated-mission assurance [13, 21, 22, 25]	Mission phases, coordination, utility, and team reliability	Markov-chain model checking, probabilistic verification, and reliability estimation	In-progress robot validation, measured robot behavior, and repairable-team analysis	Interface-carried uncertainty, provenance, software identity, lifecycle state, and recovery evidence

Continued on next page

Table 1 continued

Approach class	Protected object	Mechanism	Reported evidence	Obligations left to adjacent components
Component behavior				
Component contracts and monitors [14, 16, 26]	ROS node assumptions, guarantees, and message traces	First-order contracts, composition rules, and generated monitors; topic and service monitoring; testing guidelines	Inspection-rover and smaller examples; UAV case study; literature, repository, and practitioner studies	Team epoch, resource ownership, state age, and evidence lineage
Distributed programming and design [27, 28]	Coordination programs, proof obligations, and team architecture	Executable semantics, modular verification, generated infrastructure, and simulation-based robustness analysis	Three Koord applications; three simulated ATLAS case studies	Runtime freshness, evidence lineage, heterogeneous implementation identity, and recovery authority
Dynamical interaction				
Dynamical contracts [15, 19, 29]	Input/output trajectories, invariant sets, and temporal tasks	Satisfaction and refinement checks, compositional invariance, and decentralized synthesis	Two-vehicle, microgrid, and numerical simulations	Message age, lifecycle, implementation identity, and recovery semantics
Assume–guarantee learning [30]	Neighbor abstractions, labels, and local policies	Regular-language contracts with a lower bound on global satisfaction probability	Grid-world, room-temperature, and traffic-signal case studies	Physical and middleware validity of the assumed abstractions
Distributed state				
Middleware and QoS [8, 9, 31, 32]	Data streams, delivery status, and endpoint compatibility	QoS policies and offered/requested matching; end-to-end latency measurement across DDS implementations	Standard and architecture descriptions; configuration-specific benchmarks	Event-time validity, physical decision validity, evidence lineage, and recovery
Communication-aware coordination [18, 33, 34]	Information freshness and intermittent connectivity	Age-of-information analysis; temporal communication constraints in stochastic multi-robot planning	Critical review; analytical results; planning simulations	Implementation identity, evidence lineage, and coordinated recovery
Runtime authority				
Runtime assurance and safety filters [10, 11, 35–38]	Safety properties, safe sets, and control authority	Controller switching, barrier-constrained inputs, and learned safe control with verification	Drone surveillance in simulation and hardware; simulated multi-robot surveillance; wheeled-robot experiments; survey	Semantic evidence, implementation compatibility, and upstream reporting of interventions
Mission-level runtime verification [39, 40]	Mission feasibility and swarm-level policies	Pre-execution command rejection; compositional verdicts with an explicit unknown	Simulated driving fault scenarios; simulated swarm mission	Physical feasibility, implementation identity, and recovery orchestration
Embodied execution				
Cross-layer guardrails and runtimes [41–43]	Layered interventions, execution sessions, and evidence	Monitoring and intervention layers; compatibility preflight, session verification, and heartbeat	Position paper; simulated and physical embodiments; structured review	Common composition semantics across peer-robot interfaces

4.1 Related Surveys

Recent surveys use other organizing lenses. Li *et al.* classify language-model research for multi-robot systems into task allocation, motion planning, action generation, and human intervention [6]. Street *et al.* review formal models for robot teams under uncertainty, and Gielis *et al.* analyze how robotic applications and communication systems interact [17, 18]. Garg *et al.* organize learning-based safe control by safety notion, control method, and verification tool [38]. Kim *et al.* review safety for long-horizon manipulation by intervention locus (planning, policy, and execution) and grade the strength of the supporting evidence [43]. These surveys clarify capabilities, models, communication, or intervention layers. The present review uses a different unit of comparison: the protected object, together with the information required when its guarantee becomes another component's assumption.

4.2 Mission Intent

CROME captures mission requirements as contract-based goal graphs, checks feasibility, refines goals from a library, and synthesizes controllers [12]; later work decomposes contextual missions into context-specific sub-missions with correct-by-construction switching [23]. Contract operations can also refine a mission against a component library or repair an unrealizable specification [24]. These methods make goals, contexts, refinement, and realizability explicit at the mission boundary; data age, provenance, and implementation identity lie outside their scope.

Language-conditioned planners address an adjacent problem: turning open-ended instructions into plans, role assignments, or optimization models. RoCo coordinates language-model agents through dialogue with environment feedback, and LiP-LLM combines a language model with a predefined skill set, a dependency graph, and a linear program [4, 5]. Both provide capability evidence, while collision checking, skill definitions, and feasibility solvers remain external. Expressing intent is not the same as certifying its physical or distributed execution.

Quantitative assurance of coordinated autonomy addresses the mission level from another direction. Chaki, Dolan, and Giampapa characterize coordination phenomena and mission phases and evaluate discrete-time Markov-chain models by probabilistic model checking [13]; related work computes mission utility by probabilistic verification [21]. Reliability studies estimate the performance of sequentially coordinated missions from measured robot behavior and compare repairable team configurations [22, 25]. Within the reviewed corpus, this line is the closest mission-level predecessor of the present question because it connects coordination structure to quantitative mission evidence. Its protected object is mission or team reliability, whereas I ask which obligations must cross the heterogeneous interfaces that realize such a mission.

4.3 Component Behavior

Luckcuck *et al.* specify ROS nodes with first-order assume–guarantee contracts, compose node contracts into system properties through inference rules, and translate contracts written in the ROS Contract Language (RCL) into executable runtime monitors [14]. The approach connects behavioral contracts to node implementations; cross-robot state age, team membership, and heterogeneous deployment compatibility are not its central objects. ROSMonitoring 2.0 monitors topics and services, including message order, with partial ROS 2 support [16]. Caldas *et al.* contribute guidelines for runtime verification and field-based testing of ROS-based systems that emphasize observability, instrumentation, and reporting [26]. Together, these works show that

specification, instrumentation, and evidence collection are separate engineering concerns.

Distributed robot programming and model-driven design connect software structure to team behavior more directly. Koord provides platform-independent abstractions for distributed shared memory, sensing, and control; its executable semantics expose proof obligations, of which platform-specific ones are checked with physics-based models and hybrid verification tools [27]. ATLAS models team architecture, mission, and collective intelligence and uses simulation to explore design choices and robustness [28]. Both analyses remain tied to their languages, models, generated infrastructure, and simulation settings; neither defines an evidence envelope for every interface of a heterogeneous deployed stack.

4.4 Dynamical Interaction

Contracts for dynamical systems place assumptions on input trajectories and guarantees on outputs. Sharf *et al.* verify satisfaction and refinement of linear contracts by linear programming and prove safety in a two-vehicle example [15]. Chen *et al.* use parameterized temporal-logic contracts to compute robust invariant sets compositionally for network systems [19], and Liu *et al.* synthesize decentralized controllers for interconnected nonlinear systems from signal-temporal-logic contracts [29]. Assume–guarantee reinforcement learning expresses contracts over neighboring components as regular languages and combines local satisfaction probabilities into a lower bound for the whole system [30]. These approaches address interaction and decomposition directly, but they assume that the declared neighbor abstractions, labels, communication, dynamics, and timing are available as modeled.

4.5 Distributed State

ROS 2 provides modular robot communication through interchangeable middleware implementations, most commonly DDS [8, 9]. DDS quality-of-service (QoS) policies govern reliability, history, durability, liveliness, deadline, and lifespan. These are data-stream mechanisms; they do not establish application-level freshness or mission validity. The *Deadline* policy concerns the expected period between samples, not the age of the physical event a sample represents. The *Lifespan* policy expires samples relative to their source time stamp, which by default records the write call rather than the physical event; the standard relies on sufficiently synchronized clocks and allows a reader to substitute the reception time stamp when it detects that they are not [9].

ROS 2 latency studies show that process structure, executor behavior, and the DDS implementation affect end-to-end timing [31, 32]. Communication surveys find that coordination and networking assumptions are often designed and evaluated separately [18], and recent planning work builds temporal communication constraints and intermittent connectivity into stochastic multi-robot planning [33]. Age-of-information research characterizes how fresh the newest available update is [34]. These works provide deployment semantics and measurements rather than compositional robot contracts. They nevertheless expose a necessary bridge: a dynamical or safety guarantee that assumes current peer state needs an application-level rule that relates event time, clock uncertainty, delivery, and the physical decision.

4.6 Runtime Authority

The Simplex architecture uses a simple, verified controller to bound the behavior of a more complex one [35]. SOTER makes this pattern programmable for robotics: a decision module

transfers authority from an advanced controller to a safe controller when a monitor predicts a violation [10], and SOTER on ROS evaluates the pattern in simulated multi-robot surveillance [36]. Control barrier functions and multi-robot barrier certificates instead constrain the admissible inputs to keep a modeled safe set invariant [11, 37]. These mechanisms give strong conditional safety guarantees, but the guarantees depend on the dynamics model, state estimation, update rate, feasibility, and timely peer state. A local intervention can preserve one robot's safety while invalidating the team plan unless it is reported upstream.

Mission-level runtime verification extends monitoring beyond immediate safety. Tsai and Hariri reject driving commands that would make the mission infeasible before they execute [39]. Kekatos *et al.* compose platform, squad, and mission monitors for robot swarms over a messaging fabric that makes evidence loss observable, so that unsupported negative verdicts become an explicit *unknown* rather than an all-clear [40]. This treatment of missing evidence is the closest reviewed counterpart to the admission semantics of Section 6.3.

4.7 Embodied Execution

Kim *et al.* argue for modular safety guardrails that monitor and intervene across action, decision, and human-centered safety [41]. PhyAgentOS separates cognitive planning from physical execution and provides compatibility preflight, supervised execution sessions, evidence-grounded verification, layered safety, and heartbeat monitoring [42]. It is the closest reviewed runtime architecture to several obligations synthesized here. PhyAgentOS, however, defines one embodied-agent runtime, whereas the framework of Section 6 is an architecture-independent classification of obligations on both cross-layer and peer-robot interfaces. Within the reviewed works, I found no composition model that jointly represents temporal validity, ownership, provenance, and recovery across those interfaces.

4.8 Synthesis

The reviewed literature supplies mechanisms for mission realizability, node behavior, dynamical composition, communication observation, and runtime safety. Coverage becomes discontinuous where these mechanisms meet. A mission guarantee may end before physical feasibility; a dynamical guarantee may assume current neighbor signals; a middleware status may omit the physical event time; and a safety intervention may never return to task allocation. Section 5 turns these discontinuities into five interface gaps.

5 Where interface assumptions get lost

The reviewed approaches use different behavioral models, so their guarantees cannot be combined simply by listing them. A cross-layer execution argument must preserve the assumptions that matter when an output expressed under one model becomes an input to another. Table 2 codes each approach class against seven interface obligations, and four patterns stand out. No class is explicit and central on more than two of the seven obligations. No class treats membership, ownership, and lifecycle transitions as explicit and central. Temporal validity is explicit only in the two distributed-state classes, which treat physical validity and recovery at most partially. Recovery is explicit only where authority transfer is local to one controller or one runtime. The obligations group into five recurring gaps, G1–G5.

Table 2: Qualitative Coverage of Interface Obligations by Approach Class

	G1		G2	G3	G4		G5
Approach class	Semantic	Physical	Temporal	Uncertainty	Compatibility	Lifecycle	Recovery
Mission contracts	E	P	–	–	–	P	P
Language-conditioned planning	E	P	–	–	–	–	P
Coordinated-mission assurance	E	P	–	E	–	P	P
Component contracts and monitors	P	P	P	–	P	–	P
Distributed programming and design	E	P	P	P	E	P	P
Dynamical contracts	–	E	–	–	–	–	–
Assume–guarantee learning	P	P	–	E	–	–	–
Middleware and QoS	–	–	E	–	E	P	–
Communication-aware coordination	P	P	E	P	–	P	P
Runtime assurance and safety filters	–	E	P	P	–	–	E
Mission-level runtime verification	E	P	P	P	–	–	P
Cross-layer guardrails and runtimes	P	E	P	P	P	P	E

E: explicit and central in every source of the class; P: partial, conditional, model-internal, or present in only some sources; –: outside the stated mechanisms. Semantic: task identity, intent, and hard constraints. Physical: feasibility or safety under a physical model. Temporal: event time, data age, validity interval, and clock semantics. Uncertainty: uncertainty representation, provenance, and evidence kind. Compatibility: implementation, version, and configuration identity. Lifecycle: membership, ownership, and lifecycle transitions. Recovery: defined response to a failed check, including authority transfer. G1–G5 refer to the gaps of Section 5; sources per class are listed in Table 1.

5.1 G1: Semantic and Physical Validity

Mission contracts make goals and contexts explicit, and planning and learning systems produce assignments or actions. The receiving component still needs the task identifiers, capabilities, map, state, dynamics, resources, and communication model behind a proposal. It must also be able to distinguish a hard constraint from a planner preference, a model suggestion, and a checked feasibility claim. A meaningful plan may be physically infeasible, and a feasible plan may violate the operator's intent. Feasibility must therefore be qualified as nominal, robust, probabilistic, or conditional on a named model and operating region [4, 5, 12, 21, 24].

5.2 G2: Temporal Validity of Distributed State

Distributed execution involves several distinct times: the time of the physical event, the publication time, the arrival time, the time at which the consumer can use the data, the target time of a forecast, and the end of the validity interval. Replacing the event time with a reception time stamp hides both producer and network delay. For a stream of state updates, the age of information at time t is

$$\Delta(t) = t - u(t), \tag{2}$$

where $u(t)$ is the event time of the newest update available to the consumer [34]. Age matters only relative to a decision: the same sample can be fresh enough for a slow inspection task and too old for collision avoidance between fast robots. Comparing event times across robots also requires the clocks to be comparable. Synchronization protocols such as IEEE 1588 can bound the offset between clocks [44], but the bound helps the consumer only if it travels with the time stamp. Transport reliability or a cadence notification does not establish that the margin a

decision needs is available [9, 32]. A cross-robot interface therefore needs event-time semantics, the clock domain and an error bound, an application-specific validity bound, and a response to stale or incomparable state. Section 6.4 derives these fields from the collision constraint in (1).

5.3 G3: Uncertainty, Provenance, and Evidence

The word *uncertainty* may refer to a covariance, an interval, a reachable set, a calibrated probability, or semantic ambiguity. These representations cannot be compared or propagated in the same way, and a confidence value with no declared meaning cannot satisfy a downstream chance constraint or robust margin. The interface must identify the representation, the covered variables and correlations, the calibration regime, the validity horizon, and the propagation rule.

The same requirement applies to evidence. A consumer should be able to tell a proof from a solver status, a monitor verdict, a calibration report, a test identifier, and a deployment measurement. Provenance links a payload and its evidence to the model, configuration, observations, and transformations that produced them. Without this lineage, a downstream rejection may reveal that a claim failed but not which assumption or artifact needs repair [43, 45].

5.4 G4: Implementation and Lifecycle Compatibility

Type and wire compatibility are weaker than behavioral compatibility. Two components can decode the same message while using different coordinate frames, units, parameter sets, middleware implementations, dependency versions, or clock sources. Admitting a result therefore requires separate identities for the schema, protocol, implementation, dependencies, and configuration, together with evidence that the combination is supported [8, 9, 32].

Message contracts also omit lifecycle unless it is represented explicitly. Join, leave, restart, and partition recovery can invalidate old assignments or duplicate resource ownership. A multi-robot contract needs a team epoch, membership and ownership state, legal lifecycle transitions, and a reconciliation rule for retained or delayed data [14, 42].

5.5 G5: Intervention, Recovery, and Attribution

Monitoring is incomplete if a failed check has no defined consequence. The responsible authority may request clarification, obtain fresh state, re-estimate, replan, transfer control, degrade the mission, isolate a robot, stop, or escalate to a human. In a team, this response must be coordinated: an unreported local safety override can invalidate peer trajectories or task-completion logic. Runtime-assurance and field-testing work accordingly treats intervention, observability, and reporting as first-class mechanisms [10, 26, 36].

Fault origin and first observation must also be kept apart. A collision risk first detected by a controller may originate in semantic grounding, prediction, physical modeling, communication delay, or hardware. Recording origin, first observation, effect, and recovery authority as separate fields supports both online handling and post-mission diagnosis.

5.6 Evidence from ROS 2 Middleware

Public failure reports from the nightly continuous-integration (CI) pipeline of Nav2, the ROS 2 navigation stack, illustrate G2 and G4 in deployed software [46]. Unchanged Nav2 test sources

failed for different reasons under three ROS middleware (RMW) implementations.

- With `rmw_zenoh`, the ROS 2 middleware based on Zenoh [47], tests passed every assertion yet aborted at process exit: `rclcpp::shutdown()` ran from the destructor of a global test-fixture object after the thread-local storage of the Zenoh runtime had already been destroyed. Moving initialization and shutdown into `main()` removed the failures [48, 49].
- With Cyclone DDS, a teardown assertion in `ddsi_fini` appeared with release 11.0.1 but not with 11.0.0, using the same `rmw_cyclonedds_cpp` version. Reverting one upstream commit that changed the type-lookup request and reply headers eliminated the failure in 40 repeated runs, although that commit may expose rather than contain the defect [50].
- With Fast DDS, a service request reached its server and the callback ran, but the response was dropped because the response writer had not yet matched the client's response reader; increasing the client's wait time did not help. The behavior is consistent with a known limitation of service discovery in the Fast DDS middleware layer [46, 51].

In each case, the message types, the application code, and the declared QoS were unchanged. What differed was the implementation identity, its patch version, or the order of lifecycle events, which is exactly the information that a type-level interface does not carry (G4). The Fast DDS case also shows that the availability of one endpoint is not evidence that a complete request–response interaction can occur in time (G2). I contributed the Zenoh fix and the Cyclone DDS diagnosis; all observations are taken from the public record. These are single-host test failures rather than multi-robot field incidents, so they illustrate the gaps but do not measure their prevalence.

6 A proposed interface framework

I use the five gaps to propose a common record for information passed between components or robots. The record states what is being sent, why the receiver may use it, how to check it, and what to do if the check fails. Formal claims about refinement, realizability, controller synthesis, or the whole system still need the appropriate behavioral models from Section 4.

6.1 Interfaces as Producer–Consumer Edges

I represent the deployed system as a directed graph whose vertices are functions assigned to robots or services. A *vertical* edge $e^{\mathrm{v}} = (r, k) \to (r, k+1)$ connects consecutive functions of robot r, while a *horizontal* edge $e^{\mathrm{h}} = (r, k) \to (s, l)$ with $s \neq r$ connects robots or distributed services. A horizontal edge is more than another message path: its consumer may act on behalf of a different robot and thereby change joint feasibility, resource ownership, or a peer's safety margin. Vertical edges usually share a clock, an implementation, and a lifecycle; horizontal edges usually do not.

6.2 Interface Contracts

Definition 1 (Interface contract)**.** *For an edge $e = (p, c)$ from producer p to consumer c, an interface contract is the tuple*

$$C_e = (P_e, M_e;\ A_e, G_e, V_e, E_e, R_e), \tag{3}$$

Table 3: Interface-Contract Obligations Derived from the Five Gaps

Gap	Metadata and evidence carried by the interface	Required validators	Response on rejection
G1: Semantic and physical validity	Canonical task and resource identifiers; hard constraints; frames and units; model, operating region, and margins	Registry resolution; task, feasibility, or certificate check	Clarify, reject, repair the task, or replan
G2: Temporal validity	Event or forecast-target time; publication and availability times; validity interval; clock domain and error bound	Freshness, target-time, deadline, and clock-comparability predicates, e.g., (6)	Request fresh state, shorten the horizon, degrade, or stop
G3: Uncertainty, provenance, and evidence	Uncertainty type, scope, correlations, calibration, and propagation rule; model and solver identity; lineage; evidence kind	Uncertainty-threshold, calibration, certificate, and provenance-policy checks	Re-estimate, re-solve, choose a robust alternative, or escalate
G4: Implementation and lifecycle compatibility	Schema, protocol, implementation, dependency, and configuration identities; membership; resource ownership; team epoch	Supported-combination, epoch, ownership, and lifecycle-transition checks	Reconcile, downgrade, isolate, restart, or reallocate
G5: Intervention, recovery, and attribution	Producer and consumer authority; safety overrides; acknowledgements; completion evidence; recovery owner	Authority, acknowledgement, intervention-report, and completion predicates	Transfer authority, coordinate replanning, stop, or request human action

where P_e is a typed payload; M_e is typed metadata, such as identifiers, frames and units, times, uncertainty semantics, implementation identities, team epoch, and lineage; A_e is the consumer's assumption; G_e is the producer's guarantee; V_e is a set of validators or monitor references; E_e is the supporting evidence, such as a solver result, proof, calibration report, monitor record, or test identifier; and R_e is the response to rejection or unresolved validity.

A metadata field is contractual only when it appears in A_e, G_e, V_e, or R_e. Table 3 maps each gap of Section 5 to the metadata, validators, and recovery actions that an interface contract must specify; it is the bridge from the review to the framework. Provenance can reuse the entity–activity–agent relations of W3C PROV [45]. The framework keeps schema, protocol, and implementation identity separate because, as Section 5.6 shows, syntactic compatibility does not establish behavioral compatibility.

6.3 Compatibility and Admission

On a single edge, local compatibility requires the producer's guarantee to imply the consumer's assumption:

$$G_e \Longrightarrow A_e. \tag{4}$$

A consumer with several peer inputs must additionally check joint assumptions, such as a common team epoch, consistent frames, unique resource ownership, and a reconciliation rule after a partition; per-message checks cannot establish these relations. Condition (4) is therefore local. Proving that a composed team satisfies a global mission property requires defined input–output ownership, feedback and temporal semantics, and a composition or refinement calculus [14, 24].

Validators make the condition operational. A syntactic validator checks decoding, required fields, types, dimensions, units, frames, and schema compatibility. A semantic validator may call a solver, a runtime monitor, a perception check, a calibrated statistical test, or a certificate

verifier. Each validator returns a value in $\{\text{TRUE}, \text{FALSE}, \text{UNKNOWN}\}$. Let $\mathcal{Q}_e$ be the set of checks required on edge e, covering syntax, time, uncertainty, compatibility, assumptions, guarantees, and provenance. The edge admits the transfer at time t if and only if every required check returns TRUE:

$$\text{Admit}_e(t) \Leftrightarrow V_{e,q}(P_e, M_e, E_e, t) = \text{TRUE} \quad \forall q \in \mathcal{Q}_e. \tag{5}$$

A required check that returns UNKNOWN blocks admission, so missing metadata or evidence leads to rejection rather than to silent acceptance; Kekatos *et al.* apply the same principle to missing monitoring evidence in robot swarms [40]. Advisory checks outside $\mathcal{Q}_e$ may return UNKNOWN without blocking. Condition (5) is what turns descriptive fields into an enforced boundary: every required predicate takes part in the execution decision.

Table 4: Cross-Layer Failure-Attribution Taxonomy

Origin	Local fault	Contract concern	First observation	Possible effect	Recovery authority
Perception and estimation	Misassociation, bias, or clock error	Payload, frame, time, and uncertainty	Estimator innovation, semantic grounding, or controller residual	Wrong referent, localization drift, or unsafe state	Re-estimate, use a sensor fallback, or stop
Semantic planning	Invented relation, capability, or priority	Intent, assumptions, and guarantees	Task-model validator or downstream feasibility check	Wrong or impossible assignment; repeated replanning	Clarify, repair, or replan
Prediction	Biased or overconfident forecast	Horizon, uncertainty semantics, and validity evidence	Forecast residual or downstream feasibility check	Coordination failure or poor recovery	Re-estimate, shorten the horizon, or choose a robust plan
Physical model	Missing constraint or inaccurate contact model	Feasibility claim and solver evidence	Optimizer status, controller residual, or contact event	Slip, saturation, collision, or damage	Re-optimize, switch to a fallback controller, or stop
Runtime and network	Delay, loss, partition, version, or lifecycle fault	Age, clock, protocol, implementation identity, and assumptions	Consumer validator, missed update, exception, or teardown error	Inconsistent team state, abort, deadlock, or collision	Degrade, reconnect, reallocate, or restart
Safety function	Local or unreported intervention	Authority, intervention evidence, and recovery	Safety monitor or upstream acknowledgement	Safe local motion but failed mission or new conflict	Coordinated replanning or human escalation
Hardware	Sensor, actuator, energy, or compute fault	Capability and health guarantees	Health monitor, tracking residual, or peer timeout	Task abandonment or unsafe compensation by peers	Isolate the robot and reallocate its tasks

6.4 Temporal Admission for Peer State

The collision constraint of (1) shows which metadata a horizontal interface must carry for a temporal validator to return TRUE.

Proposition 1 (Temporal admission margin)**.** *Let robots r_i and r_j have speeds bounded by $\bar{v}_i$ and $\bar{v}_j$, and express all times in the consumer's clock. Suppose the consumer holds position estimates $\hat{p}_k$, $k \in \{i, j\}$, with stamped event times $\hat{t}_k$, such that the true event time $t_k \le t$ satisfies $|\hat{t}_k - t_k| \le \kappa_k$ and $\|\hat{p}_k - p_k(t_k)\| \le \varepsilon_k$. For a decision taken at time t and held until $t + h$, let $\bar{\Delta}_k = t + h - \hat{t}_k + \kappa_k$. If*

$$\|\hat{p}_i - \hat{p}_j\| \;\ge\; d_{\min} + \sum_{k\in\{i,j\}} (\varepsilon_k + \bar{v}_k \bar{\Delta}_k), \tag{6}$$

then $\|p_i(s) - p_j(s)\| \ge d_{\min}$ for all $s \in [t, t + h]$.

Proof. Fix $s \in [t, t+h]$ and $k \in \{i, j\}$. Because $t_k \le t \le s$ and $t_k \ge \hat{t}_k - \kappa_k$, the true age satisfies $0 \le s - t_k \le \bar{\Delta}_k$. The speed and error bounds give $\|p_k(s) - \hat{p}_k\| \le \|p_k(s) - p_k(t_k)\| + \|p_k(t_k) - \hat{p}_k\| \le \bar{v}_k \bar{\Delta}_k + \varepsilon_k$. The triangle inequality then yields $\|p_i(s) - p_j(s)\| \ge \|\hat{p}_i - \hat{p}_j\| - \sum_k (\varepsilon_k + \bar{v}_k \bar{\Delta}_k) \ge d_{\min}$. □

Condition (6) is sufficient but not necessary, and it is conservative because it ignores the direction of motion. Its value here is structural: every term on its right-hand side is a field that the interface must carry or the consumer must already know. The validator needs the event time $\hat{t}_k$ rather than the reception time, the clock-error bound κ_k, an error bound ε_k with declared meaning, and the speed bound $\bar{v}_k$. If any of these is missing, the validator returns UNKNOWN and the decision is not admitted. Substituting the reception time for $\hat{t}_k$ understates the age by the producer and network delay, which is exactly the failure described in G2.

Corollary 1 (Validity interval)**.** *Under the assumptions of Proposition 1, let $a_k = t - \hat{t}_k$ be the stamped age at decision time and*

$$m(t) = \|\hat{p}_i - \hat{p}_j\| - d_{\min} - \sum_k (\varepsilon_k + \bar{v}_k (a_k + \kappa_k))$$

the separation margin that remains at that time. Condition (6) *holds if and only if $h \le h^\star(t)$, where*

$$h^\star(t) = \frac{m(t)}{\bar{v}_i + \bar{v}_j}. \tag{7}$$

Proof. Since $\bar{\Delta}_k = a_k + \kappa_k + h$, condition (6) is equivalent to $m(t) \ge (\bar{v}_i + \bar{v}_j)\, h$, and $\bar{v}_i + \bar{v}_j > 0$. □

If $h^\star(t) < 0$, the pair cannot be admitted at time t on these estimates. Otherwise, the admitted state carries an explicit expiry time $t + h^\star(t)$, after which the consumer must refresh it or invoke R_e. The validity interval that the gaps call for is thus computed by the consumer from fields carried on the interface, not asserted by the producer.

Inference placement. Proposition 1 also prices the choice between an on-board and a remote estimate of the same peer. Suppose robot r_i can use either an on-board estimate of p_j, with error bound ε^{L} and stamped age $a^{\mathrm{L}} = t - \hat{t}^{\mathrm{L}}$ on its own clock, or a remote estimate fused from more sensors, with error bound ε^{R}, stamped age a^{R}, and clock-error bound κ^{R}. The peer term in (6) is $\varepsilon + \bar{v}_j(a + h + \kappa)$, so the on-board estimate consumes no more of the separation margin than the remote one whenever

$$\varepsilon^{\mathrm{L}} - \varepsilon^{\mathrm{R}} \;\le\; \bar{v}_j (a^{\mathrm{R}} + \kappa^{\mathrm{R}} - a^{\mathrm{L}}). \tag{8}$$

A remote estimate is worth its latency only if its accuracy advantage exceeds the distance the peer can travel during the additional age and clock uncertainty. For example, with $\bar{v}_j = 1.5\,\mathrm{m/s}$,

an additional 120 ms of age, and a 10 ms clock-error bound, the remote estimate must be at least 0.195 m more accurate. The required advantage grows linearly with peer speed, which is why on-board inference matters most for fast interactions, whereas remote inference remains attractive for slow, wide-area decisions such as task allocation [7].

Table 5: Worked Command Trace; All Thresholds Are Scenario-Specific Contract Parameters

Function	Payload P_e	Assumption A_e, guarantee G_e, metadata M_e, and evidence E_e	Validators V_e and response R_e
Semantic task formation	Roles, sector identifier, inspection goal, and corridor-clear constraint	A_e: referents resolve and two capable robots exist; G_e: confirmed hard constraints remain explicit	Resolve against map and capability registries; clarify or reject an unresolved referent
Prediction	Occupancy and joint trajectories over horizon H	M_e: event and target times, horizon, uncertainty semantics, and identity of the model of other agents	Reject or shorten H if the age exceeds $\bar{\Delta}$ or the uncertainty exceeds its declared limit $\bar{U}$
Physical grounding	Reachable viewpoints and time-parameterized paths	G_e: separation of at least $d_{\min}$ and actuator limits hold; E_e: model, solver status, and minimum margins	Re-solve, or return infeasible when the solver status is unacceptable or a required margin is negative
Distributed runtime	Assignments, paths, peer state, and acknowledgements	M_e: frame, units, schema, protocol, and implementation identities, team epoch, ownership, event times, and clock-error bounds	Admit peer state only while (6) holds; drop wrong-epoch data; reconcile, degrade, or replan when clock, ownership, or compatibility checks fail
Runtime safety	Admissible command or intervention record	G_e: conditional local-safety claim; E_e: active constraint, model, state, and update-rate assumptions, and intervention cause	Apply a feasible override and publish its cause; stop or escalate if the safety program is infeasible
Physical robots	Execution state, health, resource release, and completion evidence	G_e: capability and health; M_e: ownership and completion status	Isolate an unhealthy robot, reallocate its task, or request human action

6.5 Recovery and Lineage

A FALSE or a required UNKNOWN invokes R_e: request clarification or fresh data, re-estimate, replan, degrade, isolate, stop, or escalate. The rejection record names the failed predicate, the time, the evidence, and the responsible interface. A repaired result receives new lineage and does not silently replace the rejected claim (Fig. 2).

6.6 Failure Attribution

The same physical outcome can have several origins. A collision may begin with an incorrect semantic relation, a biased prediction, a missing physical constraint, stale peer state, or a hardware fault. Conversely, downstream checks can turn the same originating fault into a rejection, a degraded mission, or a safe stop. The taxonomy in Table 4 therefore records the origin, the contract concern, the first observation, the effect, and the recovery authority as separate fields; the first observation is the earliest interface at which evidence of the fault can become available.

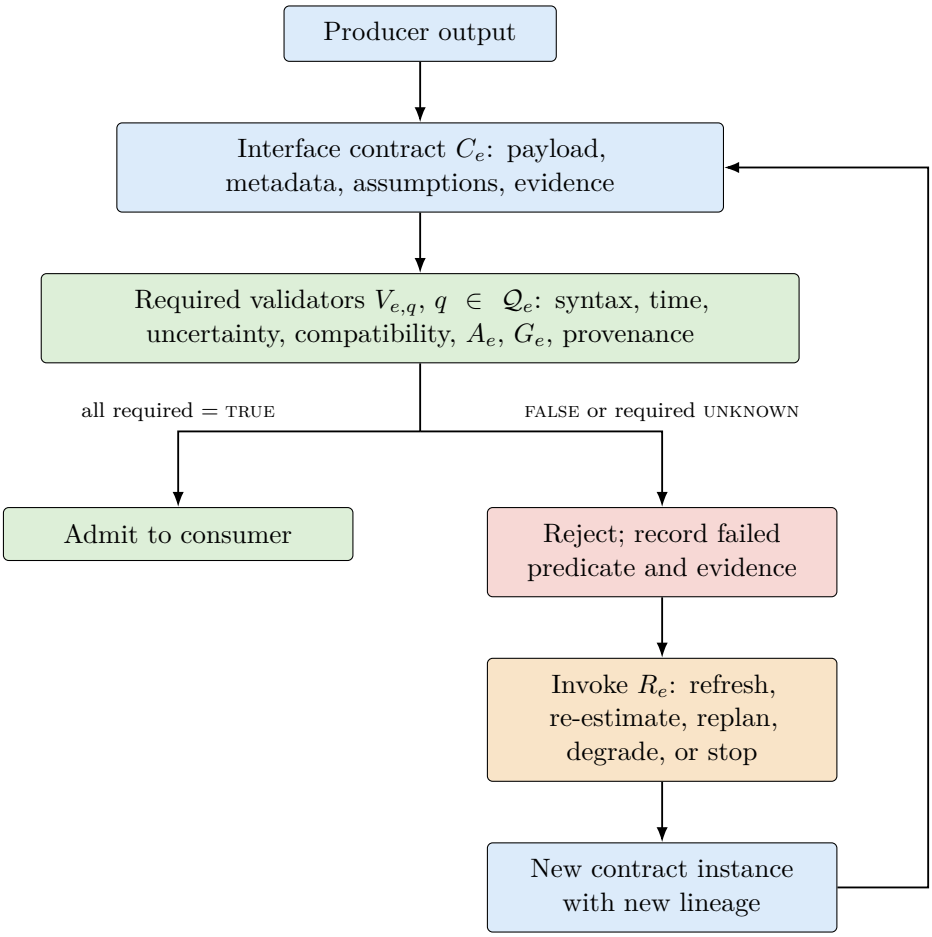


Figure 2: Admission and recovery on one interface. A required check that returns UNKNOWN blocks admission like FALSE. A successful repair creates a new contract instance and provenance record while preserving the rejected claim.

6.7 Worked Command Trace

Consider the instruction *"Use two robots to inspect sector B while keeping the corridor clear."* Table 5 follows the command from resolved roles and constraints to trajectories, runtime state, safety intervention, and completion evidence. Each row lists the information that the next validator requires in addition to the nominal output. The trace is a reference representation rather than a universal ROS message definition; thresholds and recovery policies remain scenario-specific.

7 What the findings mean

7.1 How this connects to prior work

The review leaves me with a practical question: how can assumptions and evidence survive the handoffs between planners, predictors, monitors, and safety mechanisms? The framework records those handoffs. The mission, component, dynamical, and runtime-assurance theories in Section 4 are still needed to establish the claims attached to them. The local condition (4) does not prove a global mission property. Feedback loops, shared variables, circular assumptions, realizability, probabilistic dependence, lifecycle ordering, and authority transfer require a formal semantics chosen for the implementation at hand.

Within the reviewed corpus, the coordinated-autonomy work of Chaki, Dolan, Giampapa, Porter, and colleagues is the closest mission-level predecessor because it explicitly relates robot coordination to quantitative assurance and reliability [13, 21, 22, 25]. The interface-contract framework works at a different level of abstraction: it inventories the obligations that interfaces must carry before a mission-level model, monitor, or reliability estimate can be applied consistently to a heterogeneous deployed stack. A natural connection is to treat contract rejections, recoveries, and evidence lineage as observable variables of a quantitative mission model. RCL is the closest component-level counterpart: it gives behavioral contracts formal semantics and generates monitors [14], whereas the framework supplies the inventory of payload, validity, evidence, compatibility, admission, and recovery obligations that such a formalism must instantiate. PhyAgentOS and the verification-aware swarm fabric of Kekatos *et al.* implement parts of this inventory inside one runtime or one messaging layer [40, 42]; the framework states the obligations independently of any architecture so that they can be checked on both vertical and horizontal interfaces.

7.2 Checks to try in a ROS 2 deployment

A ROS 2 developer can apply the framework by auditing every inter-robot topic, service, and action, together with the vertical interfaces that feed them. A ROS 2 message header carries a time stamp, but whether it records acquisition, processing, or publication time is a convention of the producer rather than a checked property [8]. The audit should therefore establish that peer state carries its event time and frame, that its clock domain and an offset bound are known, and that the meaning of any covariance or confidence is declared. The interface should identify the configuration, the middleware implementation and version, and the team epoch. The consumer should evaluate an admission predicate such as (6) and invoke a declared recovery action instead of continuing silently on stale or incompatible input. The failures in Section 5.6 add two lifecycle rules: initialize and shut down middleware in an explicit order rather than from

static destructors, and treat the discovery of a service as distinct from the ability to complete a request–response exchange. Not every message needs every field; the downstream component's assumptions determine which are required.

7.3 Scope of the review

I used a structured narrative search focused on the command-to-action path. The sources combine proofs, simulations, standards, issue reports, and robotics demonstrations, so the coverage matrix compares what they represent rather than estimating failure prevalence. The framework is a proposal; its effect on reliability still needs an implementation study. Security, human factors, certification, and resource-contract theory are outside this review. Proposition 1 works through peer-state admission for collision avoidance; other constraints need their own predicates.

7.4 What remains open

I see four open problems. First, the reference fields need formal semantics that compose across discrete mission logic, continuous dynamics, probabilistic prediction, and asynchronous communication. Second, lifecycle contracts must handle membership, partition recovery, ownership, and retained state without assuming a globally consistent instant. Third, uncertainty and evidence need machine-checkable types so that a consumer can decide whether a proof, a calibration result, or a deployment test supports its decision. Fourth, monitoring must scale without turning every peer exchange into an unbounded evidence log or a centralized bottleneck.

Evaluation should test the framework as an executable mechanism rather than count populated metadata fields. Relevant outcomes include false admission and false rejection, time to detection, correctness of fault attribution, recovery time, task completion, unsafe events, and runtime overhead. Reproducible studies should report the team size, platform, software and middleware versions, configuration, timing assumptions, injected faults, and run-level traces. Those measurements would show which interface checks make a practical difference.

8 What I take from this

The main lesson from this review is that a component's guarantee is hard to use when the next component cannot check its assumptions. In the coding used here, each approach class makes at most two of seven interface obligations explicit and central, and none does so for lifecycle. That pattern points to a useful place to work: the handoffs between mechanisms that already exist.

The proposed framework is a way to make those obligations visible: record what is being transferred, what makes it valid, how the receiver checks it, and who acts when a check fails. The peer-state example makes one part concrete. State age, clock error, and estimation error all consume separation margin, so a more accurate remote estimate can still be less useful than a fresher on-board one.

This gives me a concrete starting point for implementation: choose an inter-robot interface, encode the assumptions its consumer needs, and attach checks and recovery actions to them. I would start with peer-state timing and measure how those checks change stale-data admission and task completion.

References

[1] B. P. Gerkey and M. J. Matarić, "A formal analysis and taxonomy of task allocation in multi-robot systems," *The International Journal of Robotics Research*, vol. 23, no. 9, pp. 939–954, 2004.

[2] H.-L. Choi, L. Brunet, and J. P. How, "Consensus-based decentralized auctions for robust task allocation," *IEEE Transactions on Robotics*, vol. 25, no. 4, pp. 912–926, 2009.

[3] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, "Conflict-based search for optimal multi-agent pathfinding," *Artificial Intelligence*, vol. 219, pp. 40–66, 2015.

[4] Z. Mandi, S. Jain, and S. Song, "RoCo: Dialectic multi-robot collaboration with large language models," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 286–299.

[5] K. Obata, T. Aoki, T. Horii, T. Taniguchi, and T. Nagai, "LiP-LLM: Integrating linear programming and dependency graph with large language models for multi-robot task planning," *IEEE Robotics and Automation Letters*, vol. 10, no. 2, pp. 1122–1129, 2025.

[6] P. Li, Z. An, S. Abrar, and L. Zhou, "Large language models for multi-robot systems: A survey," *Autonomous Robots*, vol. 50, no. 3, 2026, Art. no. 30.

[7] B. Kehoe, S. Patil, P. Abbeel, and K. Goldberg, "A survey of research on cloud robotics and automation," *IEEE Transactions on Automation Science and Engineering*, vol. 12, no. 2, pp. 398–409, 2015.

[8] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022, Art. no. eabm6074.

[9] *Data Distribution Service (DDS), Version 1.4*, Object Management Group (OMG) Document formal/2015-04-10, 2015. [Online]. Available: https://www.omg.org/spec/DDS/1.4/

[10] A. Desai, S. Ghosh, S. A. Seshia, N. Shankar, and A. Tiwari, "SOTER: A runtime assurance framework for programming safe robotics systems," in *2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2019, pp. 138–150.

[11] L. Wang, A. D. Ames, and M. Egerstedt, "Safety barrier certificates for collisions-free multirobot systems," *IEEE Transactions on Robotics*, vol. 33, no. 3, pp. 661–674, 2017.

[12] P. Mallozzi, P. Nuzzo, P. Pelliccione, and G. Schneider, "CROME: Contract-based robotic mission specification," in *2020 18th ACM-IEEE International Conference on Formal Methods and Models for System Design (MEMOCODE)*, 2020, pp. 1–11.

[13] S. Chaki, J. M. Dolan, and J. A. Giampapa, "Toward a quantitative method for assuring coordinated autonomy," in *Proceedings of the AAMAS 2013 Workshop on Autonomous Robots and Multirobot Systems (ARMS)*, May 2013. [Online]. Available: https://publications.ri.cmu.edu/toward-a-quantitative-method-for-assuring-coordinated-autonomy

[14] M. Luckcuck, M. Farrell, A. Ferrando, R. C. Cardoso, L. A. Dennis, and M. Fisher, "A compositional approach to verifying modular robotic systems," *Robotics and Autonomous Systems*, vol. 205, 2026, Art. no. 105648.

[15] M. Sharf, B. Besselink, A. Molin, Q. Zhao, and K. H. Johansson, "Assume/guarantee contracts for dynamical systems: Theory and computational tools," *IFAC-PapersOnLine*, vol. 54, no. 5, pp. 25–30, 2021.

[16] M. G. Saadat, A. Ferrando, L. A. Dennis, and M. Fisher, "ROSMonitoring 2.0: Extending ROS runtime verification to services and ordered topics," in *Proceedings of the Sixth International Workshop on Formal Methods for Autonomous Systems (FMAS 2024)*, ser. Electronic Proceedings in Theoretical Computer Science, vol. 411, 2024, pp. 38–55.

[17] C. Street, M. Mansouri, and B. Lacerda, "Formal modelling for multi-robot systems under uncertainty," *Current Robotics Reports*, vol. 4, no. 3, pp. 55–64, 2023.

[18] J. Gielis, A. Shankar, and A. Prorok, "A critical review of communications in multi-robot systems," *Current Robotics Reports*, vol. 3, no. 4, pp. 213–225, 2022.

[19] Y. Chen, J. Anderson, K. Kalsi, S. H. Low, and A. D. Ames, "Compositional set invariance in network systems with assume-guarantee contracts," in *2019 American Control Conference (ACC)*, 2019, pp. 1027–1034.

[20] A. Benveniste, B. Caillaud, and R. Passerone, "A generic model of contracts for embedded systems," INRIA, Research Report RR-6214, 2007. [Online]. Available: https://inria.hal.science/inria-00153477

[21] S. Chaki and J. A. Giampapa, "Probabilistic verification of coordinated multi-robot missions," in *Model Checking Software (SPIN 2013)*, ser. Lecture Notes in Computer Science, vol. 7976. Springer, 2013, pp. 135–153.

[22] J. F. Porter, K. Cheung, J. A. Giampapa, and J. M. Dolan, "A reliability analysis technique for estimating sequentially coordinated multirobot mission performance," in *PRIMA 2013: Principles and Practice of Multi-Agent Systems*, ser. Lecture Notes in Computer Science, vol. 8291. Springer, 2013, pp. 276–291.

[23] P. Mallozzi, P. Nuzzo, N. Piterman, G. Schneider, and P. Pelliccione, "Correct-by-construction design of contextual robotic missions using contracts," arXiv:2306.08144, 2023.

[24] P. Mallozzi, I. Incer, P. Nuzzo, and A. Sangiovanni-Vincentelli, "Contract-based specification refinement and repair for mission planning," in *2023 IEEE/ACM 11th International Conference on Formal Methods in Software Engineering (FormaliSE)*, 2023, pp. 29–38.

[25] S. B. Stancliff, J. M. Dolan, and A. Trebi-Ollennu, "Mission reliability estimation for repairable robot teams," *International Journal of Advanced Robotic Systems*, vol. 3, no. 2, 2006, Art. no. 24.

[26] R. Caldas, J. A. P. García, M. Schiopu, P. Pelliccione, G. Rodrigues, and T. Berger, "Runtime verification and field-based testing for ROS-based robotic systems," *IEEE Transactions on Software Engineering*, vol. 50, no. 10, pp. 2544–2567, 2024.

[27] R. Ghosh, C. Hsieh, S. Misailovic, and S. Mitra, "Koord: A language for programming and verifying distributed robotics application," *Proceedings of the ACM on Programming Languages*, vol. 4, no. OOPSLA, pp. 1–30, 2020.

[28] J. Harbin, S. Gerasimou, N. Matragkas, T. Zolotas, R. Calinescu, and M. A. Santana, "Model-driven design space exploration for multi-robot systems in simulation," *Software and Systems Modeling*, vol. 22, no. 5, pp. 1665–1688, 2023.

[29] S. Liu, A. Saoud, P. Jagtap, D. V. Dimarogonas, and M. Zamani, "Compositional synthesis of signal temporal logic tasks via assume-guarantee contracts," in *2022 IEEE 61st Conference on Decision and Control (CDC)*, 2022, pp. 2184–2189.

[30] M. Kazemi, M. Perez, F. Somenzi, S. Soudjani, A. Trivedi, and A. Velasquez, "Assume-guarantee reinforcement learning," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 19, pp. 21 223–21 231, 2024.

[31] Y. Maruyama, S. Kato, and T. Azumi, "Exploring the performance of ROS2," in *Proceedings of the 13th International Conference on Embedded Software (EMSOFT)*, 2016, pp. 1–10.

[32] T. Kronauer, J. Pohlmann, M. Matthé, T. Smejkal, and G. Fettweis, "Latency analysis of ROS2 multi-node systems," in *2021 IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems (MFI)*, 2021, pp. 1–7.

[33] T. C. Silva, X. Yu, and M. A. Hsieh, "Probabilistic multi-robot planning with temporal tasks and communication constraints," *Autonomous Robots*, vol. 50, no. 1, 2026, Art. no. 2.

[34] S. Kaul, R. Yates, and M. Gruteser, "Real-time status: How often should one update?" in *2012 Proceedings IEEE INFOCOM*, 2012, pp. 2731–2735.

[35] L. Sha, "Using simplicity to control complexity," *IEEE Software*, vol. 18, no. 4, pp. 20–28, 2001.

[36] S. Shivakumar, H. Torfah, A. Desai, and S. A. Seshia, "SOTER on ROS: A run-time assurance framework on the robot operating system," in *Runtime Verification (RV 2020)*, ser. Lecture Notes in Computer Science, vol. 12399. Springer, 2020, pp. 184–194.

[37] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017.

[38] K. Garg, S. Zhang, O. So, C. Dawson, and C. Fan, "Learning safe control for multi-robot systems: Methods, verification, and open challenges," *Annual Reviews in Control*, vol. 57, 2024, Art. no. 100948.

[39] C. Tsai and S. Hariri, "Mission-level runtime assurance framework for autonomous driving," arXiv:2606.06996, 2026.

[40] N. Kekatos, P. Katsaros, A. Lekidis, T. Nestoridis, and T. Nianios, "Mission-level runtime assurance for LLM-assisted ISR swarms over a verification-aware fabric," arXiv:2607.23532, 2026.

[41] J. Kim, W. Chen, D. Soleymanzadeh, Y. Ding, X. Gao, Z. Tu, R. Zhang, F. Fei, S. Veer, Y. Lyu, M. Zheng, and Y. Gu, "Modular safety guardrails are necessary for foundation-model-enabled robots in the real world," arXiv:2602.04056, 2026.

[42] Y. Liu, W. Chen, X. Song, T. Pu, S. Mo, Y. Bai, Z. Chen, Q. Sun, L. Zhong, Y. Shen, and L. Lin, "PhyAgentOS: A self-evolving operating system for embodied agents with decoupled cognitive planning and physical execution," arXiv:2607.16636, 2026.

[43] D. Kim, D. Park, S. Lee, J. Kim, Y. Oh, J. Shin, and S. Yoon, "Safe embodied AI for long-horizon tasks: A cross-layer analysis of robotic manipulation," arXiv:2606.05660, 2026.

[44] *IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems*, IEEE Std 1588-2019, 2020.

[45] L. Moreau and P. Missier, *PROV-DM: The PROV Data Model*, World Wide Web Consortium W3C Recommendation, 2013. [Online]. Available: https://www.w3.org/TR/prov-dm/

[46] "Nightly CI failing across multiple RMWs," GitHub issue ros-navigation/navigation2 #6163, 2026. [Online]. Available: https://github.com/ros-navigation/navigation2/issues/6163

[47] A. Corsaro, L. Cominardi, O. Hecart, G. Baldoni, J. E. P. Avital, J. Loudet, C. Guimarães, M. Ilyin, and D. Bannov, "Zenoh: Unifying communication, storage and computation from the cloud to the microcontroller," in *2023 26th Euromicro Conference on Digital System Design (DSD)*, 2023, pp. 422–428.

[48] "Use main function instead of global variable to setup the test environment," GitHub pull request ros-navigation/navigation2 #4948, 2025. [Online]. Available: https://github.com/ros-navigation/navigation2/pull/4948

[49] "Fix Zenoh test failures caused by late cleanup," GitHub pull request ros-navigation/navigation2 #6241, 2026. [Online]. Available: https://github.com/ros-navigation/navigation2/pull/6241

[50] "Cyclone DDS 11.0.1 intermittently asserts in ddsi_fini during ROS 2 action teardown," GitHub issue eclipse-cyclonedds/cyclonedds #2426, 2026. [Online]. Available: https://github.com/eclipse-cyclonedds/cyclonedds/issues/2426

[51] "Improve service discovery," GitHub issue ros2/rmw_fastrtps #392, 2020. [Online]. Available: https://github.com/ros2/rmw_fastrtps/issues/392